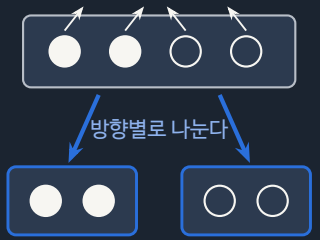
ASWISS

Abstraction Split WIth Sampling by Strategy

업데이트 방향이 갈리는 곳에서 추상화를 나누는 방법



작은 화살표: 각 정보집합의 업데이트 방향

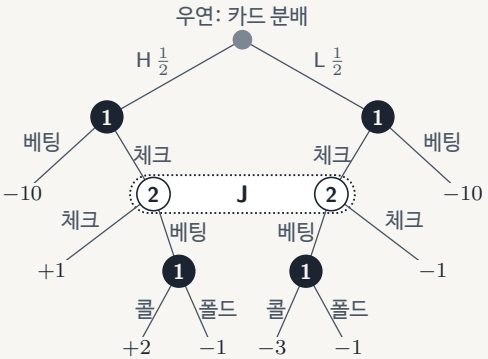
졸업 논문 연구 · 2인 영합 불완전 정보 게임의 적응형 추상화 · 2026년 11월



오늘 발표할 연구는 ASWISS, Abstraction Split WIth Sampling by Strategy입니다.
한 문장으로 말하면, 미리 정한 특징으로 게임을 묶어 두는 대신, 학습 중인 전략이 정보집합마다 어느 쪽으로 움직이려 하는지, 즉 업데이트 방향을 보고, 한 묶음 안에서 방향이 갈리는 곳만 그 자리에서 나누는 추상화 방법입니다.
오른쪽 그림이 그 그림입니다. 위의 묶음 안에서 두 정보집합은 오른쪽으로, 두 정보집합은 왼쪽으로 움직이려 합니다. 방향이 갈리니 그 묶음을 방향별로 둘로 나눕니다.

이 규칙만으로, 표준 학습기를 쓸 때 원 게임의 균형으로 수렴한다는 것이 이 연구의 중심 정리입니다. 발표는 여덟 가지 질문에 차례로 답하는 순서로 갑니다.

불완전 정보 게임과 정보집합



플레이어 2는 카드를 모른다:
두 상태가 하나의 정보집합 J

게임 트리 · 카드 분배(우연)와 두 플레이어의 행동. 끝의 수는 플레이어 1의 보수(칩)

보는 것 / 못 보는 것 · 플레이어 1(●)은 자기 카드를 본다. 플레이어 2(○)는 행동만 본다

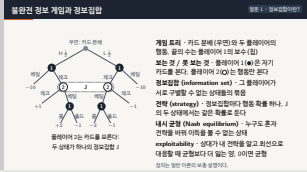
정보집합 (information set) · 그 플레이어가 서로 구별할 수 없는 상태들의 묶음

전략 (strategy) · 정보집합마다 행동 확률 하나. J의 두 상태에서는 같은 확률로 둔다

내시 균형 (Nash equilibrium) · 누구도 혼자 전략을 바꿔 이득을 볼 수 없는 상태

exploitability · 상대가 내 전략을 알고 최선으로 대응할 때 균형보다 더 잃는 양. 0이면 균형

정의는 일반 이론의 보충 설명이다.



먼저 불완전 정보 게임이 무엇인지 이 작은 포커로 보겠습니다. 이 게임은 뒤에서 ASWISS의 동작을 설명할 때도 계속 쓰는 예시입니다.

맨 위 회색 점은 우연, 즉 카드 분배입니다. 플레이어 1이 H 또는 L 카드를 절반씩 받습니다. 검은 원은 플레이어 1, 흰 원은 플레이어 2의 차례이고, 맨 아래 숫자는 플레이어 1이 얻는 칩입니다.

핵심은 플레이어 2가 카드를 보지 못한다는 것입니다. 플레이어 1이 H를 들고 체크했는지 L을 들고 체크했는지 구별할 수 없으므로, 점선으로 묶은 두 상태가 플레이어 2에게는 같은 상황입니다. 이렇게 구별할 수 없는 상태의 묶음을 정보집합이라고 합니다.

전략은 상태마다가 아니라 정보집합마다 정합니다. 그래서 J의 두 상태에서 플레이어 2는 같은 확률로 체크하거나 베팅해야 합니다.

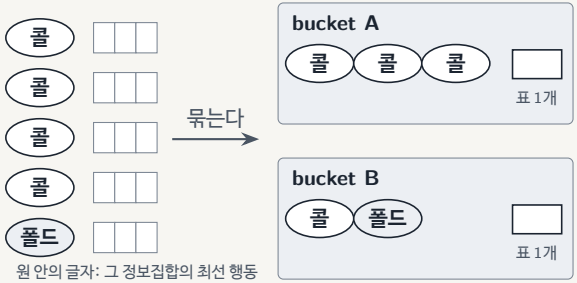
내시 균형은 누구도 혼자 전략을 바꿔서 이득을 볼 수 없는 상태입니다. exploitability는 상대가 내 전략을 알고 최선으로 대응했을 때 균형보다 얼마나 더 잃는지를 재는 값으로, 0이면 균형입니다. 이 값이 이후 실험의 기본 지표입니다.

참고로 이 게임에서 균형은 H면 콜, L이면 폴드, 플레이어 2는 체크이고, 플레이어 1의 값은 0입니다.

추상화: 묶으면 얻는 것과 잃는 것

큰 게임은 정보집합이 너무 많다. 세븐 포커에서는 같은 정보집합이 학습 중 거의 다시 나오지 않아 정보집합마다 표를 둘 수 없다.

정보집합마다 표



원 안의 글자: 그 정보집합의 최선 행동

✓ 이득

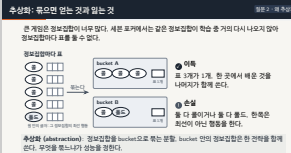
표 3개가 1개. 한 곳에서 배운 것을 나머지가 함께 쓴다.

! 손실

둘 다 콜이거나 둘 다 폴드. 한쪽은 최선이 아닌 행동을 한다.

추상화 (abstraction): 정보집합을 bucket으로 묶는 분할. bucket 안의 정보집합은 한 전략을 함께 쓴다. 무엇을 묶느냐가 성능을 정한다.

추상화: 묶으면 얻는 것과 잃는 것



왜 추상화가 필요한지부터 보겠습니다. 실제 크기의 세븐 포커는 정보집합이 너무 많아서, 학습하는 동안 같은 정보집합을 다시 만나는 일이 거의 없습니다. 정보집합마다 따로 표를 두면 표마다 배울 경험이 없습니다.

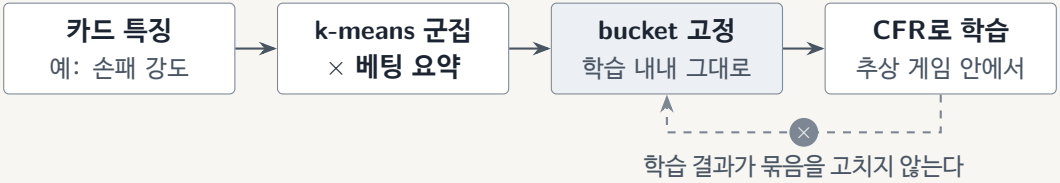
그래서 비슷한 정보집합을 bucket으로 묶고, 묶음 하나가 전략 표 하나를 함께 씁니다. 이것이 추상화입니다.

위의 bucket A처럼 모두 같은 행동을 원하는 정보집합을 묶으면 손해가 없습니다. 표가 줄고, 한 곳에서 배운 것을 나머지가 함께 씁니다.

그러나 아래 bucket B처럼 콜을 원하는 것과 폴드를 원하는 것을 묶으면, 표가 하나뿐이라 둘 중 하나는 최선이 아닌 행동을 하게 됩니다. 이것이 추상화의 손실입니다.

결국 질문은 무엇을 묶고 무엇을 나눌 것인가입니다.

고정 추상화에 빠져 있는 것



❶ 묶는 기준이 해를 보지 않는다

무엇을 합쳐도 되는지는 게임의 해(균형)가 정한다. 카드 특징은 그것을 모른다.

❷ 전략적 동등성의 보장이 없다

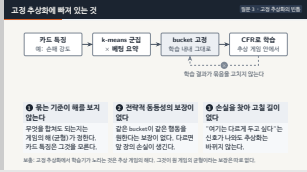
같은 bucket이 같은 행동을 원한다는 보장이 없다. 다르면 앞 장의 손실이 생긴다.

❸ 손실을 찾아 고칠 길이 없다

"여기는 다르게 두고 싶다"는 신호가 나와도 추상화는 바뀌지 않는다.

보충: 고정 추상화에서 학습기가 노리는 것은 추상 게임의 해다. 그것이 원 게임의 균형이라는 보장은 따로 없다.

고정 추상화에 빠져 있는 것

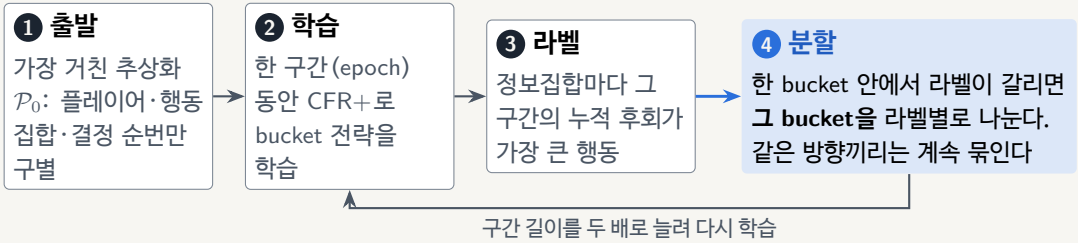


지금까지 큰 게임에서 쓰던 방식은 고정 추상화입니다. 카드 강도 같은 특징을 k-means로 군집하고, 베팅 상황을 요약해서 bucket을 만든 뒤, 그 bucket을 학습 내내 고정해 두고 CFR로 풉니다.
여기에는 세 가지가 빠져 있습니다. 첫째, 묶는 기준이 게임의 해를 보지 않습니다. 둘째, 그래서 같은 bucket 안의 정보집합들이 같은 행동을 원한다는 보장, 즉 전략적 동등성의 보장이 없습니다. 셋째, 학습 도중에 "여기는 다르게 두고 싶다"는 신호가 나와도 추상화를 고칠 통로가 없습니다.

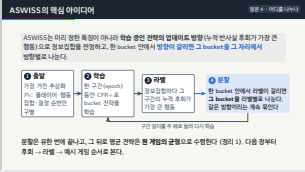
ASWISS는 바로 이 끊어진 화살표를 잇는 방법입니다. 학습이 내는 신호, 즉 업데이트 방향으로 묶음을 고칩니다.

ASWISS의 핵심 아이디어

ASWISS는 미리 정한 특징이 아니라 **학습 중인 전략의 업데이트 방향**(누적 반사실 후회가 가장 큰 행동)으로 정보집합을 판정하고, 한 bucket 안에서 **방향이 갈리면 그 bucket을 그 자리에서** 방향별로 나눈다.



분할은 유한 번에 끝나고, 그 뒤로 평균 전략은 **원 게임의 균형**으로 수렴한다 (정리 1). 다음 장부터 후회 → 라벨 → 예시 게임 순서로 본다.



이것이 이 연구의 핵심입니다. ASWISS는 미리 정한 특징이 아니라, 학습 중인 전략의 업데이트 방향으로 정보집합을 판정하고, 한 bucket 안에서 방향이 갈리면 그 bucket을 그 자리에서 방향별로 나눕니다.

절차는 네 단계입니다. 가장 거친 추상화에서 시작해, 한 구간 동안 CFR+로 학습합니다. 그 구간 동안 각 정보집합에 쌓인 반사실 후회를 보고, 가장 큰 후회를 가진 행동을 라벨로 붙입니다. 이 라벨이 그 정보집합의 업데이트 방향, 즉 regret matching이 확률을 올리려는 방향입니다.

한 bucket 안에서 라벨이 갈리면 그 bucket을 라벨별로 나눕니다. 같은 방향을 원하는 정보집합끼리는 계속 묶여 있으므로 압축은 유지됩니다. 앞 결정의 bucket은 건드리지 않습니다.

구간 길이를 두 배씩 늘리며 이것을 반복하면, 분할은 유한 번에 끝나고 그 뒤로 평균 전략이 원 게임의 균형으로 수렴합니다.

이것이 정리 1입니다.

후회: 그 행동을 했더라면 얼마나 나았을까

예시 게임의 결정 2 (체크 → 상대 베팅 → 콜 또는 폴드). 지금 전략이 "항상 콜"인 한 반복을 보자.

카드	콜의 값	폴드의 값	폴드의 후회	$\times\frac{1}{2}$
H	$+2$	-1	$-1-(+2)=-3$	$\mathbf{-1.5}$
L	-3	-1	$-1-(-3)=+2$	$\mathbf{+1.0}$

후회 (regret) = 그 행동의 값 − 지금 전략의 값. 양수면 "바꾸고 싶다".

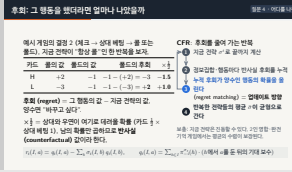
$\times\frac{1}{2}$ = 상대와 우연이 여기로 데려올 확률 (카드 $\frac{1}{2}\times$ 상대 베팅 1). 남의 확률만 곱하므로 **반사실 (counterfactual)** 값이라 한다.

CFR: 후회를 줄여 가는 반복

1. 지금 전략 σ^t로 끝까지 계산
2. 정보집합·행동마다 반사실 후회를 누적
3. **누적 후회가 양수인 행동의 확률을 올린다** (regret matching) = **업데이트 방향**
4. **반복한 전략들의 평균 $\bar{\sigma}$이 균형으로 간다**

보충: 지금 전략은 진동할 수 있다. 2인 영합·완전 기억 게임에서는 평균의 수렴이 보장된다.

$r_t(I,a)=q_t(I,a)-\sum_b \sigma_t(I,b)\,q_t(I,b)$, $\quad q_t(I,a)=\sum_{h\in I}\pi_{-i}^{\sigma_t}(h)\cdot(h$에서 a를 둔 뒤의 기대 보수$)$

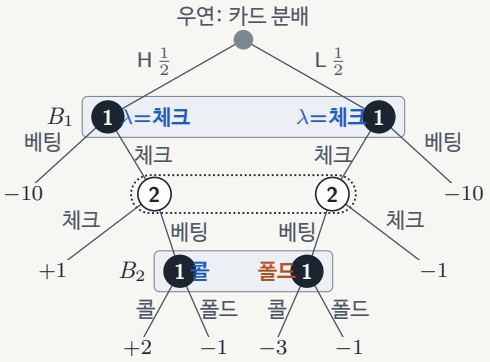


분할을 판단하는 재료는 후회입니다. 후회는 "그 행동을 했더라면 얼마나 나왔을까"입니다.
예시 게임의 두 번째 결정에서, 지금 전략이 항상 콜인 한 반복을 보겠습니다. H를 들고 있으면 콜이 $+2$, 폴드가 -1이라 폴드의 후회는 -3입니다. L을 들고 있으면 콜이 -3, 폴드가 -1이라 폴드의 후회는 $+2$, 즉 폴드로 바꾸고 싶습니다. 여기에 상대와 우연이 이 상태로 데려올 확률 $1/2$을 곱한 것이 반사실 후회입니다. H는 -1.5, L은 $+1.0$입니다.
CFR은 이 후회를 반복마다 누적하고, 누적 후회가 양수인 행동의 확률을 그 크기에 비례해 올립니다. 그러니 누적 후회가 가장 큰 행동이 바로 학습이 그 정보집합을 움직이려는 방향, 즉 업데이트 방향입니다. ASWISS는 이 방향을 라벨로 씁니다.

지금 전략은 진동할 수 있지만, 반복한 전략들의 평균은 2인 영합 완전 기억 게임에서 균형으로 수렴한다는 것이 알려져 있습니다.

우리가 내놓는 정책도 평균 전략입니다.

라벨: 정보집합마다의 업데이트 방향



출발점 $\mathcal{P}_0$: (플레이어, 행동집합, 자기 결정 순번)이 같은 정보집합을 하나로 묶는 가장 거친 분할. 이 게임에서는 B_1, B_2, $\{J\}$

1 정보집합 I · 라벨을 매기는 단위

bucket B · 전략을 함께 쓰는 정보집합 묶음

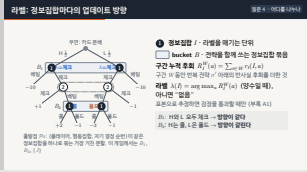
구간 누적 후회 $R_I^W(a) = \sum_{t \in W} r_t(I, a)$

구간 W동안 반복 전략 σ^t 아래의 반사실 후회를 더한 것

라벨 $\lambda(I) = \arg\max_a R_I^W(a)$ **(양수일 때),**
아니면 "없음"

표본으로 추정하면 검정을 통과할 때만 (부록 A1)

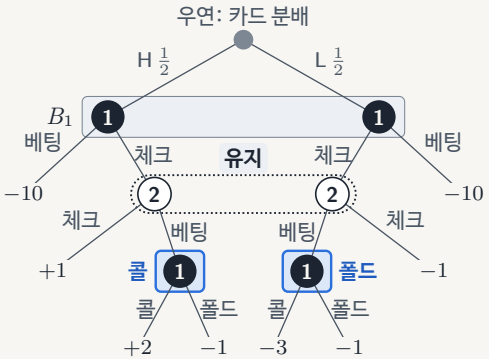
B_1: H와 L 모두 체크 → **방향이 같다**
B_2: H는 콜, L은 폴드 → **방향이 갈린다**



이제 같은 게임 위에서 ASWISS가 보는 것을 정리하겠습니다. 처음에는 가장 거친 분할 P0에서 시작합니다. 같은 플레이어, 같은 행동집합, 같은 결정 순번이면 하나로 묶습니다. 그래서 첫 결정의 H와 L이 B1, 두 번째 결정의 H와 L이 B2입니다.
전략은 bucket마다 하나지만, 라벨은 정보집합마다 붙입니다. 한 구간 동안 각 정보집합에 쌓인 반사실 후회를 보고, 가장 큰 행동을 라벨로 삼습니다. 양수가 아니면 바꿀 이유가 없으니 "없음"입니다.
첫 결정에서는 베팅하면 -10이라 H와 L 모두 체크가 최선이고, 라벨도 둘 다 체크입니다. 방향이 같습니다.

두 번째 결정에서는 앞 장에서 본 것처럼 H는 콜을, L은 폴드를 원합니다. B2는 하나의 전략을 쓰기 때문에, 학습하는 동안 H에서는 콜 쪽, L에서는 폴드 쪽 후회가 쌓입니다. 방향이 갈립니다.

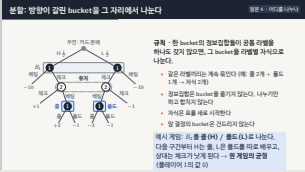
분할: 방향이 갈린 bucket을 그 자리에서 나눈다



규칙 · 한 bucket의 정보집합들이 공통 라벨을 하나도 갖지 않으면, 그 bucket을 라벨별 자식으로 나눈다.

- 같은 라벨끼리는 계속 묶인다 (예: 콜 2개 + 폴드 1개 → 자식 2개)
- 정보집합은 bucket을 옮기지 않는다. 나누기만 하고 합치지 않는다
- 자식은 표를 새로 시작한다
- 앞 결정의 bucket은 건드리지 않는다

예시 게임: B_2를 **콜(H) / 폴드(L)**로 나눈다. 다음 구간부터 H는 콜, L은 폴드를 따로 배우고, 상대는 체크가 낫게 된다 → **원 게임의 균형** (플레이어 1의 값 0)



두 번째 결정의 B2는 라벨이 콜과 폴드로 갈렸습니다. 공통 라벨이 없으므로 B2를 라벨별로 나눕니다. H는 콜 자식으로, L은 폴드 자식으로 갑니다. 첫 결정의 B1은 라벨이 같으니 그대로 둡니다.

규칙의 성질 네 가지입니다. 같은 라벨끼리는 계속 묶여 있으므로, 예를 들어 콜을 원하는 정보집합이 여럿이면 그들은 한 자식에 함께 남습니다. 정보집합은 bucket을 옮기지 않고, 나누기만 하고 합치지는 않습니다. 새 자식은 표를 새로 시작합니다. 그리고 앞 결정의 bucket은 건드리지 않습니다. 문제가 생긴 그 자리에서만 나눕니다.

이렇게 나누면 다음 구간부터 H는 콜, L은 폴드를 따로 배웁니다. 그러면 플레이어 2는 베팅보다 체크가 낫고, 원 게임의 균형에 도달합니다.

참고로, 라벨을 정보집합이 아니라 bucket 이력이라는 더 거친 단위로 모으면 B2의 H와 L이 같은 키가 되어 이 차이가 보이지 않습니다. 그 경우를 위해 만든 역방향 분할 같은 장치들은 부록의 트릭으로 옮겼습니다.

소속: 정보집합은 어느 bucket으로 가는가

작은 게임: 표로 기록한다

분할할 때 각 정보집합이 받은 라벨로 자식을 정하고, 정보집합 → bucket 표에 기록한다. 이후 그 정보집합은 늘 그 bucket의 전략을 쓴다.

정리 1이 다루는 경우다. mini-stud(51,114개), Leduc(288개)에서 그대로 쓴다.

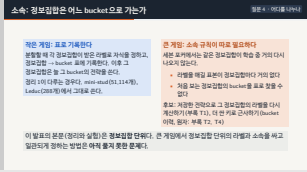
큰 게임: 소속 규칙이 따로 필요하다

세븐 포커에서는 같은 정보집합이 학습 중 거의 다시 나오지 않는다.

- 라벨을 매길 표본이 정보집합마다 거의 없다
- 처음 보는 정보집합의 bucket을 표로 찾을 수 없다

후보: 저장한 전략으로 그 정보집합의 라벨을 다시 계산하기(부록 T1), 더 싼 키로 근사하기(bucket 이력, 원자: 부록 T2, T4)

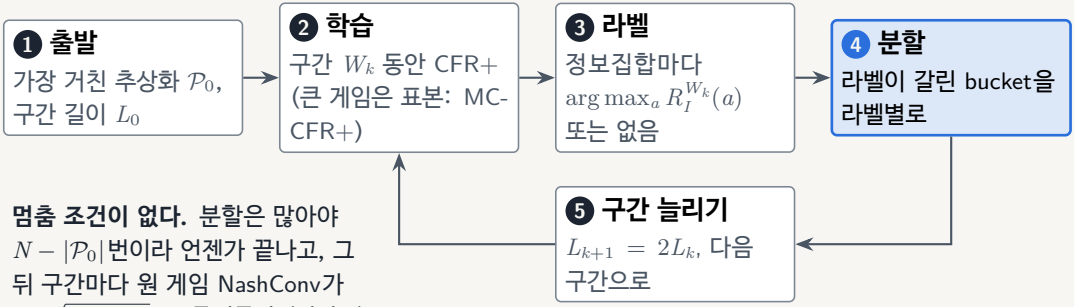
이 발표의 본문(정리와 실험)은 **정보집합 단위**다. 큰 게임에서 정보집합 단위의 라벨과 소속을 싸고 일관되게 정하는 방법은 **아직 풀지 못한 문제**다.



분할한 뒤에는, 각 정보집합이 어느 자식 bucket의 전략을 쓰는지 정해야 합니다. 이것을 소속이라고 하겠습니다.
작은 게임에서는 간단합니다. 분할할 때 각 정보집합이 받은 라벨로 자식을 정하고 표에 적어 두면 됩니다. 정리 1이 다루는 경우가 이것이고, mini-stud와 Leduc 실험이 이렇게 했습니다.
큰 게임에서는 두 가지가 막힙니다. 같은 정보집합이 거의 다시 나오지 않으므로 정보집합마다 라벨을 매길 표본이 없고, 처음 보는 정보집합이 어느 bucket인지 표로 찾을 수도 없습니다. 그래서 소속을 정하는 규칙이 따로 필요합니다. 저장해 둔 전략으로 라벨을 다시 계산하는 방법과, bucket 이력이나 카드 군집 같은 싼 키로 근사하는 방법을 시도했고, 이것들은 부록의 트릭으로 정리했습니다.

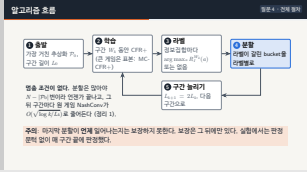
본문의 정리와 실험은 모두 정보집합 단위입니다. 큰 게임에서 이것을 어떻게 싸고 일관되게 할지는 아직 풀지 못한 문제로 남겨 둡니다.

알고리즘 흐름



멈춤 조건이 없다. 분할은 많아야 $N - |\mathcal{P}_0|$번이라 언젠가 끝나고, 그 뒤 구간마다 원 게임 NashConv가 $O(\sqrt{\log k / L_k})$로 줄어든다 (정리 1).

주의: 마지막 분할이 **언제** 일어나는지는 보장하지 못한다. 보장은 그 뒤에만 있다. 실험에서는 판정 문턱 없이 매 구간 끝에 판정했다.



지금까지의 내용을 하나의 절차로 모으면 이렇습니다.
1번, 가장 거친 추상화에서 출발합니다. 2번, 한 구간 동안 CFR+로 학습합니다. 큰 게임에서는 표본을 쓰는 MCCFR+입니다. 3번, 그 구간 동안 각 정보집합에 쌓인 후회로 라벨을 붙입니다. 4번, 라벨이 갈린 bucket을 라벨별로 나눕니다. 5번, 구간 길이를 두 배로 늘리고 다시 학습합니다.
이 알고리즘에는 따로 멈추는 조건이 없습니다. 분할은 bucket을 쪼개기만 하므로 많아야 정보집합 수만큼만 일어나고, 그래서 언젠가 끝납니다. 그 뒤로는 구간마다 원 게임의 NashConv가 줄어 0으로 갑니다. 이것이 정리 1입니다.

다만 마지막 분할이 언제 일어나는지는 보장하지 못합니다. 보장은 그 뒤에만 있습니다.

의사코드와 정의

$\mathcal{P} \leftarrow \mathcal{P}_0, \quad L \leftarrow L_0$
for $k = 0, 1, 2, \ldots$
  구간 W_k의 L반복 동안 CFR+로 학습하며
    정보집합마다 $R_I^{W_k} = \sum_{t \in W_k} r_t(I)$를 쌓는다
  for each 정보집합 I:
    $\lambda(I) \leftarrow \arg\max_a R_I^{W_k}(a)$ 또는 없음
  for each bucket B: 라벨이 둘 이상이면
    라벨별 자식으로 나눈다
  $L \leftarrow 2L$
출력: 마지막 구간의 평균 전략 $\bar{\sigma}^{(k)}$

$\mathcal{P}_0$ · (플레이어, 행동집합, 자기 결정 순번)이 같은 정보집합을 하나로

학습기 · bucket마다 RM+(CFR+). 표본이면 외부 샘플링

검정 · 표본 누적값이 잡음보다 클 때만 라벨을 준다 (예: $\tilde{R}(a^*) > z\sqrt{\sum \tilde{r}^2}$). 정확 계산이면 작은 문턱만

출력 정책 · 구간 동안 전략들의 실현 계획 평균 (자기 도달확률로 가중)



같은 절차를 의사코드로 쓰면 이렇습니다. 논문에서 증명하는 기본 형태입니다.
분할을 P0로, 구간 길이를 L0로 두고 시작합니다. 구간마다 CFR+로 학습하면서 각 정보집합의 후회를 그 구간 동안 따로 더해 둡니다. 구간이 끝나면 정보집합마다 라벨을 붙이고, 라벨이 둘 이상 섞인 bucket을 라벨별로 나눕니다. 구간 길이를 두 배로 늘리고 다시 갑니다.

표본으로 학습하면 누적 후회에 잡음이 있으므로, 잡음보다 확실히 클 때만 라벨을 줍니다. 출력하는 정책은 그 구간 동안 전략들의 평균이고, 자기 도달확률로 가중한 평균입니다.

원 게임 NashConv를 두 항으로 나눈다

보조정리 1·2 (구간 W의 평균 전략 $\bar{\sigma}^W$, 길이 $L = |W|$)

$$\mathrm{NashConv}(\bar{\sigma}^W) \;\le\; \frac{1}{L}\Big[\sum_B \psi(R_B^W) \;+\; \sum_B H_B^W\Big]$$

bucket 후회 $\psi(R_B)$,
$\psi(x) = \max(0, \max_a x_a)$
bucket이 하나의 정보집합처럼 느끼는 구간 후회

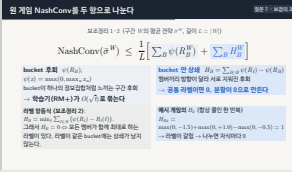
→ **학습기 (RM+)가 $O(\sqrt{t})$로 묶는다**

bucket 안 상쇄 $H_B = \sum_{I\in B}\psi(R_I) - \psi(R_B)$
멤버끼리 방향이 달라 서로 지워진 후회

→ **공통 라벨이면 0, 분할이 0으로 만든다**

라벨 항등식 (보조정리 2):
$H_B = \min_\ell \sum_{I\in B}\big(\psi(R_I) - R_I(\ell)\big)$.
그래서 $H_B = 0 \Leftrightarrow$ 모든 멤버가 함께 최대로 하는 라벨이 있다. 라벨이 같은 bucket에는 상쇄가 남지 않는다.

예시 게임의 B_2 (항상 콜인 한 반복)
$H_{B_2} =$
$\max(0,-1.5)+\max(0,+1.0)-\max(0,-0.5) = 1$
→ 라벨이 갈림 → 나누면 자식마다 0



이론의 출발점은 이 분해입니다. 구간 평균 전략의 원 게임 NashConv는 두 항의 합으로 위에서 막힙니다.
첫째 항은 bucket 후회입니다. bucket을 하나의 정보집합처럼 보았을 때 그 구간 동안 쌓인 후회로, CFR+의 RM+가 반복 수의 제곱근 이하로 묶습니다.
둘째 항은 bucket 안 상쇄입니다. 멤버들의 후회를 각자 더한 것과, 합쳐서 본 bucket의 후회의 차이입니다. 멤버들이 서로 다른 방향을 원해서 합치면 지워지는 양입니다.
핵심은 아래 왼쪽의 라벨 항등식입니다. 이 상쇄량은 정확히 "모든 멤버를 한 라벨에 맞출 때의 최소 비용"과 같습니다. 그래서 모든 멤버가 함께 최대로 하는 라벨이 있으면 상쇄는 정확히 0입니다. 라벨이 같은 bucket에는 숨은 손실이 없다는 뜻이고, 라벨이 갈린 bucket만 나누면 된다는 근거입니다.

예시 게임의 B2에 넣어 보면 상쇄가 1이고, 실제 NashConv도 1입니다. 라벨이 갈려 나누면 자식마다 상쇄가 0이 됩니다.

정리 1: 분할이 끝나면 원 게임 균형으로 수렴한다

정리 1 외부 샘플링 CFR+(동시 갱신), 두 배씩 늘어나는 구간, 문턱 $\theta = o(L)$인 라벨 검정을 쓰면, 확률 $1-\delta$ 이상으로

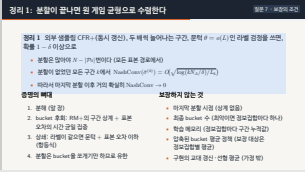
- 분할은 많아야 $N - |\mathcal{P}_0|$번이다 (모든 표본 경로에서)
- 분할이 없었던 모든 구간 k에서 $\mathrm{NashConv}(\bar{\sigma}^{(k)}) = O\big(\sqrt{\log(kN_A/\delta)/L_k}\big)$
- 따라서 마지막 분할 이후 거의 확실히 $\mathrm{NashConv} \to 0$

증명의 뼈대

1. 분해 (앞 장)
2. bucket 후회: RM+의 구간 상계 + 표본 오차의 시간 균일 집중
3. 상쇄: 라벨이 같으면 문턱 + 표본 오차 이하 (항등식)
4. 분할은 bucket을 쪼개기만 하므로 유한

보장하지 않는 것

- 마지막 분할 시점 (상계 없음)
- 최종 bucket 수 (최악이면 정보집합마다 하나)
- 학습 메모리 (정보집합마다 구간 누적값)
- 압축된 bucket 평균 정책 (보장 대상은 정보집합별 평균)
- 구현의 교대 갱신·선형 평균 (가정 밖)


정리 1: 분할이 끝나면 원 게임 균형으로 수렴한다
정리 1 외부 샘플링 CFR+(정시 경신), 두 배력 놀이나는 구간, 문턱 $\theta = \alpha(L)$인 라벨 검정을 쓰면, 확률 $1-\delta$ 이상으로
분할은 많아야 $N - |\mathcal{P}_1|$번이다 (모든 표본 경로에서)
분할이 없었던 모든 구간 k에서 $\mathrm{NashConv}(\pi^{(t)}) = O\left(\sqrt{\log(|\mathcal{A}|_{\max}/\delta)/L_k}\right)$
따라서 마지막 분할 이후 가장 확실히 $\mathrm{NashConv} \to 0$.
증명의 뼈대
1. 분해 (알 값)
2. bucket 증회: RM+의 구간 상계 + 표본 오차의 시간 균일 검증
3. 상세: 라벨이 갈라진 순간 + 표본 오차 이하 (합집시)
4. 분할은 bucket 물 쪼개기만 하므로 유한
보장해지 않는 것
마지막 분할 시점 (상계 없음)
최종 bucket 수 (최악이면 정보집합마다 하나)
학습 메모리 (정보집합마다 구간 누적값)
압축된 bucket 평균 경계 (표준 대상은 정보집합별 평균)
구현의 교대 경신·선형 평균 (가정 밖)
정보 7 / 보장의 조건

중심 정리입니다. 표준 학습기인 외부 샘플링 CFR+를 쓰고, 구간을 두 배씩 늘리고, 잡음을 고려한 라벨 검정을 쓰면, 높은 확률로 다음이 성립합니다.

첫째, 분할은 유한 번입니다. 둘째, 분할이 없었던 구간마다 원 게임의 NashConv가 구간 길이의 제곱근에 반비례해 줄어듭니다. 그래서 마지막 분할 이후에는 0으로 수렴합니다. 속도는 추상화 없는 CFR+의 상계와 로그 인자를 빼면 같습니다.

증명은 네 단계입니다. 앞 장의 분해, RM+가 bucket 후회를 묶는다는 것, 라벨이 같은 bucket의 상쇄가 작다는 항등식, 그리고 분할의 유한성입니다. 학습기에 대한 추가 조건이 없다는 것이 중요합니다.

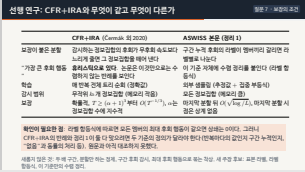
보장하지 않는 것도 분명히 하겠습니다. 마지막 분할이 언제인지, 최종 bucket이 몇 개인지는 말하지 못합니다. 라벨을 위해 정보집합마다 구간 누적값을 저장해야 하므로 학습 중 메모리는 줄지 않습니다. 보장하는 정책은 정보집합별 평균이고, bucket 표로 압축한 평균은 아닙니다. 그리고 실제 구현은 교대 갱신과 선형 평균을 써서 정리의 가정과 조금 다릅니다.

선행 연구: CFR+IRA와 무엇이 같고 무엇이 다른가

	CFR+IRA (Čermák 외 2020)	**ASWISS 본문 (정리 1)**
보장이 붙은 분할	감시하는 정보집합의 후회가 무후회 속도보다 느리게 줄면 그 정보집합을 떼어 낸다	구간 누적 후회의 라벨이 멤버끼리 갈리면 라벨별로 나눈다
"가장 큰 후회 행동"	**휴리스틱으로 있다**. 논문은 이것만으로는 수렴하지 않는 반례를 보인다	이 기준 자체에 수렴 정리를 붙인다 (라벨 항등식)
학습	매 반복 전체 트리 순회 (정확값)	외부 샘플링 (추정값 + 집중 부등식)
감시 범위	무작위 k_b개 정보집합 (메모리 작음)	모든 정보집합 (메모리 큼)
보장	확률적, $T \geq (\alpha+1)^3$ 부터 $O(T^{-1/3})$, α는 정보집합 수에 지수적	마지막 분할 뒤 $O(\sqrt{\log / L})$, 마지막 분할 시점은 상계 없음

확인이 필요한 점: 라벨 항등식에 따르면 모든 멤버의 최대 후회 행동이 같으면 상쇄는 0이다. 그러니 CFR+IRA의 반례와 정리 1이 둘 다 맞으려면 두 기준의 정의가 달라야 한다 (반복마다의 값인지 구간 누적인지, "없음"과 동률의 처리 등). 원문과 아직 대조하지 못했다.

새롭지 않은 것: 두 배 구간, 분할만 하는 정제, 구간 후회 감시, 최대 후회 행동으로 묶는 착상. 새 주장 후보: 표본 라벨, 라벨 항등식, 이 기준만의 수렴 정리.


선행 연구: CFR+IRA와 무엇이 같고 무엇이 다른가

	CFR+IRA (Čermák 외 2020)	ASWISS 본문 (정의 1)
보장이 붙은 분할	감시하는 정보집합의 후회가 무후회 속도보다 느리게 줄면 그 정보집합을 떼어 낸다	구간 누적 후회의 라벨이 멤버까리 같아면 라벨로 시능다
"가장 큰 후회 행동	휴리스틱으로 얽다. 논문은 이것만으로는 수렴하지 않는 반례를 보인다	이 기준 자체에 수렴 정리를 붙인다 (라벨 할 증사)
학습	매 반복 전체 트리 순회 (정확값)	외부 샘플링 (추정값 + 검증 부등식)
감시 범위	무후회 노개 정보집합 (매시리 작음)	모든 정보집합 (메모리 큼)
보장	확률적, $T \geq (\alpha+1)^2$ 부터 $O(T^{-1/2})$, α는 정보집합 수에 지수적	마지막 본할 뒤 $O(\sqrt{\log T}/T)$, 마지막 본할 시점은 상계 없음

확인에 필요한 점. 각별 정점사에 따르면 모든 멤버의 최대 후회 합동이 같으면 상세는 0이다. 그러나 CFR+IRA의 반례와 같이 나 둘 다 맞고라면 두 기준이 정의가 달라야 합니다. 예를 들어 반복마다의 후회를 보는지 구간 누적을 보는지, 없음 라벨과 동률을 어떻게 다루는지입니다. 원문의 정확한 정의와 반례는 아직 직접 대조하지 못했고, 논문에 쓰기 전에 반드시 확인하겠습니다.

새롭지 않은 것: 루 배 구간, 분할만 하는 정제, 구간 후회 감시. 최대 후회 합동으로 묶는 작성. 세 수정 후보: 표준 라벨, 각별 합동식, 이 기준만의 수렴 정리.

이 방법과 가장 가까운 선행 연구는 Čermák, Lisý, Bošanský의 CFR+IRA입니다. CFR+IRA도 학습하면서 후회를 보고 추상화를 나눕니다.

CFR+IRA에서 수렴 보장이 붙은 분할 기준은, 감시하는 정보집합의 후회가 무후회 알고리즘의 속도보다 느리게 줄어들면 그 정보집합을 떼어 내는 것입니다. 그리고 가장 큰 후회를 가진 행동으로 정보집합을 묶는 휴리스틱도 함께 씁니다. 그 휴리스틱이 우리 라벨과 같은 계열입니다. 논문은 그 휴리스틱만으로는 수렴하지 않는 반례를 보였습니다.

우리는 바로 그 계열의 기준에 수렴 정리를 붙입니다. 근거는 라벨 항등식입니다. 그러니 두 결과가 모두 맞으려면 두 기준의 정의가 달라야 합니다. 예를 들어 반복마다의 후회를 보는지 구간 누적을 보는지, 없음 라벨과 동률을 어떻게 다루는지입니다. 원문의 정확한 정의와 반례는 아직 직접 대조하지 못했고, 논문에 쓰기 전에 반드시 확인하겠습니다.

그 밖의 차이는 학습 방식입니다. CFR+IRA는 매 반복 전체 트리를 순회하고, 우리는 외부 샘플링의 추정값으로 판정합니다.

대신 CFR+IRA는 일부 정보집합만 감시해서 메모리가 작고, 우리는 모든 정보집합을 추적해서 메모리가 큽니다. 두 배 구간,

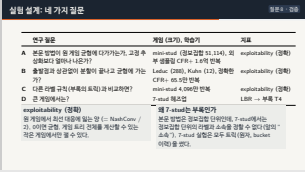
분할만 하는 정제, 구간 후회 감시는 CFR+IRA에 이미 있으므로 새로운 점으로 주장하지 않습니다.

실험 설계: 네 가지 질문

	연구 질문	게임 (크기), 학습기	지표
A	본문 방법이 원 게임 균형에 다가가는가, 고정 추상화보다 얼마나 나은가?	mini-stud (정보집합 51,114), 외부 샘플링 CFR+ 1.6억 반복	exploitability (정확)
B	출발점과 상관없이 분할이 끝나고 균형에 가는가?	Leduc (288), Kuhn (12), 정확한 CFR+ 65.5만 반복	exploitability (정확)
C	다른 라벨 규칙 (부록의 트릭)과 비교하면?	mini-stud 4,096만 반복	exploitability (정확)
D	큰 게임에서는?	7-stud 헤즈업	LBR → 부록 T4

exploitability (정확)
원 게임에서 최선 대응에 잃는 양 (= NashConv / 2). 0이면 균형. 게임 트리 전체를 계산할 수 있는 작은 게임에서만 잴 수 있다.

왜 7-stud는 부록인가
본문 방법은 정보집합 단위인데, 7-stud에서는 정보집합 단위의 라벨과 소속을 정할 수 없다 (앞의 "소속"). 7-stud 실험은 모두 트릭 (원자, bucket 이력)을 썼다.

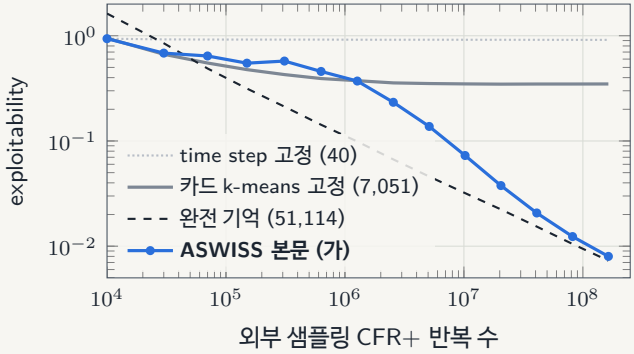


실험은 네 가지 질문에 답하도록 정리했습니다.
A는 mini-stud에서, 본문 방법이 원 게임의 균형에 얼마나 다가가는지, 고정 추상화와 비교해 어떤지 봅니다. B는 Leduc과 Kuhn에서, 가장 거친 분할이든 무작위 추상화든 어떤 출발점에서도 분할이 끝나고 균형에 가는지 봅니다. 정리 1이 말하는 상황이 실제로 나오는지 확인하는 실험입니다. C는 같은 mini-stud에서 다른 라벨 규칙들, 즉 부록에 트릭으로 정리한 방법들과 비교합니다.

작은 게임에서는 게임 트리 전체를 계산할 수 있어서 정확한 exploitability를 씁니다. 7-stud는 정보집합 단위의 라벨과 소속을 정할 수 없어서 모든 실험이 트릭을 썼습니다. 그래서 7-stud 결과는 부록 T4로 옮겼습니다.

A. mini-stud: 완전 기억에 다가가고, 고정 추상화는 멈춘다

질문 A · 본문 방법이 원 게임 균형에 다가가는가, 고정 추상화보다 얼마나 나은가?



seed 1 · bucket 평균 정책의 정확한 exploitability (낮을수록 좋음) · 괄호: bucket 수 (고정) · 두 축 모두 로그

1.10배

1.638억 반복: 0.0080 대 완전 기억 0.0073. bucket 26,844개 (정보집합의 53%)

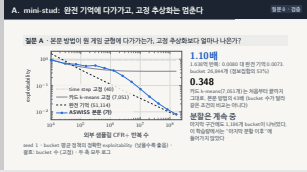
0.348

카드 k-means(7,051개)는 처음부터 끝까지 그대로. 본문 방법의 43배 (bucket 수가 달라 같은 조건의 비교는 아니다)

분할은 계속 중

마지막 구간에도 1,186개 bucket이 나뉘었다. 이 학습량에서는 "마지막 분할 이후"에 들어가지 않았다

└A. mini-stud: 완전 기억에 다가가고, 고정 추상화는 멈춘다



첫 번째 질문입니다. mini-stud는 정보집합이 51,114개라 모두 열거할 수 있어서 정확한 exploitability를 잴 수 있습니다. 외부 샘플링 CFR+로 1.6억 반복을 돌렸고, 두 축 모두 로그 축입니다.

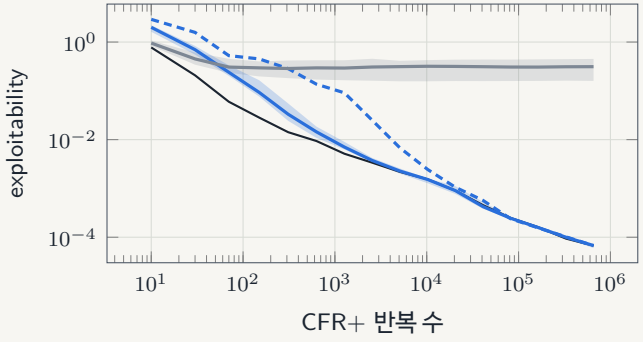
파란 선이 본문 방법입니다. 검은 파선인 완전 기억, 즉 추상화 없이 정보집합마다 표를 둔 경우에 점점 붙어서, 1.638억 반복에서 0.0080 대 0.0073, 1.10배가 되었습니다. 그때 bucket은 26,844개로 정보집합의 53퍼센트입니다.

회색 실선은 카드 k-means 고정 추상화로, 처음부터 끝까지 0.348에 머뭅니다. 학습을 아무리 해도 추상화가 숨긴 손실이 남기 때문입니다. 다만 k-means는 bucket이 7,051개로 수가 달라서, 같은 조건의 비교는 아닙니다.

솔직하게 덧붙이면, 이 게임에서는 1.6억 반복 동안에도 분할이 끝나지 않았습니다. 마지막 구간에서도 천여 개 bucket이 나뉘었습니다. 정리 1의 수렴 속도 보장은 마지막 분할 이후에만 성립하므로, 이 곡선의 속도 자체를 정리 1이 설명하지는 않습니다.

B. Leduc: 어떤 출발점에서도 분할이 끝나고 균형에 갔다

질문 B · 가장 거친 분할이든 무작위 추상화든, 분할이 끝나고 원 게임 균형에 가는가?



정확한 CFR+ · 정보집합 288 · 띠: 최소–최대 · k-means는 각 실행의 최종 bucket 수에 맞춘 고정 추상화를 같은 반복 수만큼 학습

추상화 없는 CFR+

본문 방법, 무작위 출발 20개 (중앙값)

본문 방법, 가장 거친 $\mathcal{P}_0$ 에서

같은 bucket 수의 k-means (중앙값)

21 / 21 분할 종료

마지막 분할은 모두 16.4만 반복까지 ($\mathcal{P}_0$ 는 4.1만), 그 뒤로 없음

0.0000600–0.0000702

추상화 없는 CFR+는 0.0000673

222–270 / 288

최종 bucket. 같은 수의 k-means 0.160–0.898. Kuhn은 21/21이 12개로 나뉘어 같은 값

└B. Leduc: 어떤 출발점에서도 분할이 끝나고 균형에 갔다



두 번째 질문은 정리 1이 말하는 상황, 즉 분할이 끝나고 그 뒤로 균형에 가는 것이 실제로 나오는지입니다. Leduc 홀덤은 정보집합이 288개라 정확한 CFR+와 정확한 exploitability를 쓸 수 있습니다.

가장 거친 분할 P0과 무작위 추상화 20개, 모두 21개 출발점에서 돌렸습니다. 21개 모두 16만 4천 반복 이전에 마지막 분할이 일어났고, 그 뒤 65만 5천 반복까지 더 나뉘지 않았습니다. 정리 1의 "마지막 분할 이후" 구간에 들어간 것입니다.

그 뒤 exploitability는 0.0000600에서 0.0000702로, 추상화 없는 CFR+의 0.0000673과 같은 수준입니다. 최종 bucket은 288개 중 222에서 270개였습니다. 정보집합을 다 나누지 않고도 같은 값에 왔습니다.

회색은 각 실행의 최종 bucket 수에 맞춘 k-means 고정 추상화를 같은 반복 수만큼 학습한 기준선으로, 0.160에서 0.898에 머뭅니다. Kuhn에서는 21개 모두 150반복 안에 정보집합 12개를 모두 나누고 추상화 없는 CFR+와 같은 값에 갔습니다.

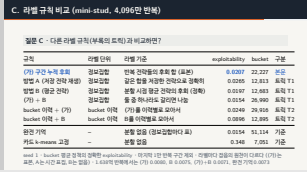
이것은 이 게임, 이 설정에서 관찰한 결과입니다. 정리 1은 마지막 분할이 언제인지 말하지 않으므로, 16만 반복이라는 시점은 관찰일 뿐입니다.

C. 라벨 규칙 비교 (mini-stud, 4,096만 반복)

질문 C · 다른 라벨 규칙(부록의 트릭)과 비교하면?

규칙	라벨 단위	라벨 기준	exploitability	bucket	구분
(가) 구간 누적 후회	정보집합	반복 전략들의 후회 합 (표본)	**0.0207**	22,227	본문
방법 A (저장 전략 재생)	정보집합	같은 합을 저장한 전략으로 정확히	0.0265	12,813	트릭 T1
방법 B (평균 전략)	정보집합	분할 시점 평균 전략의 후회 (정확)	0.0197	12,683	트릭 T1
(가) + B	정보집합	둘 중 하나라도 갈리면 나눔	0.0154	26,990	트릭 T1
bucket 이력 + (가)	bucket 이력	(가)를 이력별로 모아서	0.0249	29,916	트릭 T2
bucket 이력 + B	bucket 이력	B를 이력별로 모아서	0.0896	12,895	트릭 T2
완전 기억	–	분할 없음 (정보집합마다 표)	0.0154	51,114	기준
카드 k-means 고정	–	분할 없음	0.348	7,051	기준

seed 1 · bucket 평균 정책의 정확한 exploitability · 마지막 1만 반복 구간 제외 · 라벨마다 잡음의 원천이 다르다 ((가)는 표본, A는 시간 표집, B는 없음) · 1.638억 반복에서는 (가) 0.0080, B 0.0075, (가)+B 0.0071, 완전 기억 0.0073


C. 라벨 규칙 비교 (mini-stud, 4,096만 반복)

질문 C · 다른 라벨 규칙(부록의 트릭)과 비교하면?

규칙	라벨 단위	라벨 기준	exploitability	bucket	구분
(가) 구간 누적 후회	정보집합	반복 전략들의 후회 합 (표본)	0.0207	22,227	본문
방법 A (사전 전략 배정)	정보집합	같은 합을 계정한 전략으로 정확히	0.0265	12,813	트릭 T1
방법 B (평균 전략)	정보집합	분할 시점 평균 전략의 후회 (정확)	0.0197	12,683	트릭 T1
(가) + B	정보집합	둘 중 하나라도 같이면 나눔	0.0154	26,990	트릭 T1
bucket 이력 + (가)	bucket 이력	(가)를 이력별로 모아서	0.0246	29,916	트릭 T2
bucket 이력 + B	bucket 이력	B를 이력별로 모아서	0.0896	12,895	트릭 T2
완전 기억	–	분할 없음 (정보집합마다 표)	0.0154	51,114	기준
카드 k-means 고정	–	분할 없음	0.348	7,051	기준

seed 1 · bucket 평균 정확의 정확한 exploitability · 마지막 1만 반복 구간 제외 · 라벨bucket 없음의 원천이 다르다 (가)는 표본, A는 사전 표집 B는 없음) · 1.6억만 반복하면 (가) 0.0080, B 0.0075, (가)+B 0.0077, 완전 기억 0.0073

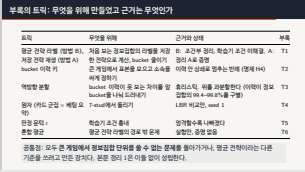
세 번째 질문은 다른 라벨 규칙과의 비교입니다. 같은 mini-stud, 같은 seed, 같은 일정으로 돌렸습니다.
본문 방법 (가)는 0.0207, bucket 22,227개였습니다. 평균 전략으로 라벨을 매기는 방법 B는 0.0197로 비슷한 값에 bucket이 절반 남짓이었고, 둘을 함께 쓰면 완전 기억과 같은 0.0154였습니다. 이런 이점이 있어서 B는 부록의 첫 번째 트릭으로 정리했습니다. 다만 B는 이론이 조건부이고, B의 라벨은 정확값이라 잡음이 없다는 점에서 (가)와 같은 조건의 비교가 아닙니다. 라벨을 정보집합이 아니라 bucket 이력이라는 더 거친 단위로 모으면 나빠졌고, 특히 B와 함께 쓰면 0.0896으로 크게 나빠졌습니다. 이력 안에서 정보집합끼리 후회가 상쇄되기 때문이고, 이것이 역방향 분할이라는 보완을 만들게 된 이유입니다. 이 내용은 부록 T2와 T3에 있습니다.

더 길게 1.6억 반복까지 돌리면 세 규칙 모두 완전 기억과 거의 같아집니다.

부록의 트릭: 무엇을 위해 만들었고 근거는 무엇인가

트릭	무엇을 위해	근거와 상태	부록
평균 전략 라벨 (방법 B), 저장 전략 재생 (방법 A)	처음 보는 정보집합의 라벨을 저장한 전략으로 계산, bucket 줄이기	B: 조건부 정리, 학습기 조건 미해결. A: 정리 A로 증명	T1
bucket 이력 키	큰 게임에서 표본을 모으고 소속을 싸게 정하기	이력 안 상쇄로 멈추는 반례 (명제 H4)	T2
역방향 분할	bucket 이력이 못 보는 차이를 앞 bucket을 나눠 드러내기	휴리스틱. 위를 과분할한다 (이력이 정보집합의 99.4–99.8%를 구별)	T3
원자 (카드 군집 $\times$ 베팅 요약)	7-stud에서 돌리기	LBR 비교만, seed 1	T4
판정 문턱 ε	학습기 조건 흉내	엄격할수록 나빠졌다	T5
혼합 평균	평균 전략 라벨의 경로 밖 문제	실험만, 증명 없음	T6

공통점: 모두 **큰 게임에서 정보집합 단위를 쓸 수 없는 문제**를 돌아가거나, 평균 전략이라는 다른 기준을 쓰려고 만든 장치다. 본문 정리 1은 이들 없이 성립한다.


부록의 트릭: 무엇을 위해 만들었고 근거는 무엇인가

트릭	무엇을 위해	근거와 상태	부록
평균 전략 라벨 (방법 B), 저경 전략 재생 (방법 A)	처음 보는 정보집합의 라벨을 저장한 전략으로 계산, bucket 줄이기	B: 조건부 정리, 학습기 조건 미해결, A: 정리 A로 증명	T1
bucket 이력 키	큰 게임에서 표본을 모으고 소속을 바꿔 정하기	이력 안 상태로 맞추는 반례 (명제 H4)	T2
역방향 분할	bucket 이력이 곧 보는 차이를 알 bucket을 나눔 (식 6-2-B 참)	흔히스딕, 위를 과분할한다 (이력이 정보 집합의 H-4 - H-3 스를 구별)	T3
원자 (카드 군집 × 베팅 요약)	7-stud에서 돌리기	LBR 비교만, seed 1	T4
판정 문턱 ε	학습기 조건 줄내	엄격할수록 나빠졌다	T5
혼합 평균	평균 전략 라벨의 경로 밖 문제	실험만, 증명 없음	T6

공통점: 모두 큰 게임에서 정보집합 단위를 쓸 수 없는 문제를 돌아가거나 다른 기준을 쓰려고 만든 장치라는 것입니다. 본문 정리 1은 이들 없이 성립한다.

지금까지 시도한 다른 장치들은 모두 부록의 트릭으로 정리했습니다. 각각 무엇을 위해 만들었는지와 근거를 표로 모았습니다.
평균 전략 라벨은 실험에서 bucket을 줄였지만 이론은 조건부입니다. bucket 이력은 큰 게임에서 표본을 모으려고 만든 키인데, 이력 안에서 정보집합끼리 상쇄되어 멈추는 반례가 있습니다. 역방향 분할은 그 반례를 메우려고 앞 결정의 bucket을 나누는 장치로, 아래 bucket의 문제를 위해 위 bucket을 과분할합니다. mini-stud에서는 결국 이력이 정보집합을 거의 다 구별하게 되었습니다.
원자는 7-stud에서 돌리기 위한 카드 군집과 베팅 요약이고, 판정 문턱은 이론의 학습기 조건을 흉내 낸 것인데 엄격할수록 나빠졌습니다. 혼합 평균은 평균 전략 라벨의 경로 밖 문제를 피하는 장치입니다.

공통점은, 모두 큰 게임에서 정보집합 단위를 쓸 수 없는 문제를 돌아가거나 다른 기준을 쓰려고 만든 장치라는 것입니다. 본문의 정리 1은 이들 없이 성립합니다.

검증된 것과 아직 아닌 것

확인된 것

✓ **증명 (초안)** · NashConv 분해와 라벨 항등식, 유한 분할, 정리 1: 외부 샘플링 CFR+로 마지막 분할 뒤 원 게임 균형에 수렴

✓ **Leduc · Kuhn** · 21개 출발점 모두 분할이 끝나고 추상화 없는 CFR+와 같은 값 (Leduc bucket 222–270/288)

✓ **mini-stud** · 1.6억 반복에서 완전 기억의 1.10배, bucket 53%. 고정 k-means는 0.348에 머묾

아직 아닌 것

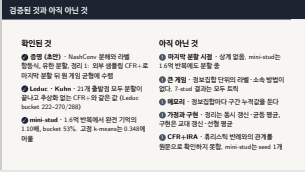
! **마지막 분할 시점** · 상계 없음. mini-stud는 1.6억 반복에도 분할 중

! **큰 게임** · 정보집합 단위의 라벨·소속 방법이 없다. 7-stud 결과는 모두 트릭

! **메모리** · 정보집합마다 구간 누적값을 둔다

! **가정과 구현** · 정리는 동시 갱신·균등 평균, 구현은 교대 갱신·선형 평균

! **CFR+IRA** · 휴리스틱 반례와의 관계를 원문으로 확인하지 못함. mini-stud는 seed 1개



검증된 것과 아직 아닌 것을 나누겠습니다.

확인된 것입니다. 이론에서는 NashConv 분해와 라벨 항등식, 분할의 유한성, 그리고 외부 샘플링 CFR+로 마지막 분할 뒤 원 게임 균형에 수렴한다는 정리 1이 증명되어 있습니다. 다만 아직 다른 사람이 검토하지 않은 초안입니다. 실험에서는 Leduc과 Kuhn의 21개 출발점이 모두 분할을 마치고 추상화 없는 CFR+와 같은 값에 갔고, mini-stud에서는 완전 기억의 1.1배에 bucket 53퍼센트로 다가갔습니다.

아직 아닌 것입니다. 마지막 분할이 언제인지 보장하지 못하고, mini-stud에서는 1.6억 반복에도 분할이 계속되었습니다. 큰 게임에서 정보집합 단위로 라벨과 소속을 정하는 방법이 없어서 7-stud 결과는 모두 트릭에 기댄 것입니다. 라벨을 위해 정보집합마다 누적값을 저장하므로 메모리는 줄지 않습니다. 정리의 가정과 구현이 조금 다르고, CFR+IRA 휴리스틱의 반례와의 관계는 원문으로 확인해야 합니다.

정리와 다음 단계

학습 중인 전략의 **업데이트 방향**이 한 bucket 안에서 갈리는 곳만 **그 자리에서** 나눈다. 이 규칙만으로 표준 학습기가 원 게임 균형에 수렴한다.

요약

- 라벨 항등식: 라벨이 같은 bucket에는 숨은 손실이 없다
- 정리 1: 분할은 유한하고, 그 뒤 원 게임 균형에 수렴
- 작은 게임: 모든 출발점에서 분할이 끝나고 추상화 없는 CFR+와 같은 값

다음 단계

1. CFR+IRA 원문과 대조 (휴리스틱의 정의와 반례)
2. 큰 게임에서 정보집합 단위의 라벨과 소속 정하기
3. 마지막 분할 시점의 상계 (여유 조건 아래)
4. 학습 메모리 줄이기 (저장 전략 재생의 대가 줄이기)



정리하겠습니다. ASWISS는 학습 중인 전략의 업데이트 방향이 한 bucket 안에서 갈리는 곳만, 그 자리에서 나누는 방법입니다. 라벨 항등식 덕분에 라벨이 같은 bucket에는 숨은 손실이 없고, 그래서 이 규칙만으로 표준 학습기가 원 게임의 균형에 수렴한다는 것을 보였습니다. 작은 게임에서는 모든 출발점에서 분할이 끝나고 추상화 없는 CFR+와 같은 값에 갔습니다.

다음 단계는 네 가지입니다. 먼저 CFR+IRA 원문과 대조해서, 최대 후회 행동 휴리스틱의 정확한 정의와 반례가 우리 정리와 어떻게 다른지 확인합니다. 둘째, 큰 게임에서 정보집합 단위의 라벨과 소속을 정하는 방법입니다. 셋째, 마지막 분할 시점의 상계입니다. 넷째, 학습 중 메모리를 줄이는 것입니다.

감사합니다. 정리의 세부와 트릭들은 부록에 있습니다.

A1. 기호와 정의 (본문 방법)

반복 t의 반사실 후회 $r_t(I,a) = q_t(I,a) - \sum_b \sigma_t(I,b)\, q_t(I,b)$, $q_t(I,a) = \sum_{h\in I} \pi_{-i}^{\sigma_t}(h) \sum_{z \sqsupseteq ha} \pi^{\sigma_t}(ha,z)\, u_i(z)$

구간 누적 후회 $W = (s,t]$, $L = |W|$: $R_I^W(a) = \sum_{u\in W} r_u(I,a)$, **표본 추정** $\tilde{R}_I^W$ (외부 샘플링, 방문하지 않으면 0)

bucket 합 $R_B^W = \sum_{I\in B} R_I^W$, $\psi(x) = \max(0, \max_a x_a)$

라벨 $\hat{a}_I = \arg\max_a \tilde{R}_I^W(a)$. $\tilde{R}_I^W(\hat{a}_I) > \theta_I^W$이면 $\lambda(I) = \hat{a}_I$, 아니면 $\lambda(I) = \varnothing$ ("없음")

문턱 조건 (T) $0 \le \theta_I^W \le \Delta_I \kappa(W)$, $\kappa(W) = o(|W|)$. 예: 정규 검정 $z\sqrt{\sum \tilde{r}^2} \le z\Delta_I\sqrt{|W|}$

분할 판정 때 멤버 라벨이 두 종류 이상인 bucket을 라벨별로 나눈다. 자식은 $Q = 0$, 평균 표 0에서 시작. 병합 없음

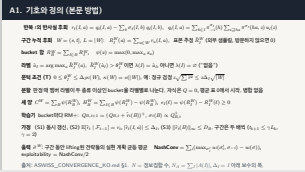
세 양 $C^W = \sum_B \psi(R_B^W)$, $H_B^W = \sum_{I\in B} \psi(R_I^W) - \psi(R_B^W)$, $e_I(\ell) = \psi(R_I^W) - R_I^W(\ell) \ge 0$

학습기 bucket마다 RM+: $Q_{B,t+1} = (Q_{B,t} + \tilde{r}_t(B))^+$, $\sigma_t(B) \propto Q_{B,t}^+$

가정 (S1) 동시 갱신, (S2) $\mathbb{E}[\tilde{r}_t \mid \mathcal{F}_{t-1}] = r_t$, $|\tilde{r}_t(I,a)| \le \Delta_I$, (S3) $\|\tilde{r}_t(B)\|_\infty \le D_B$. 구간은 두 배씩 ($t_{k+1} \le \gamma L_k$, $\gamma = 2$)

출력 $\bar{\sigma}^W$: 구간 동안 lifting된 전략들의 실현 계획 균등 평균 **NashConv** $= \sum_i (\max_{\sigma_i'} u_i(\sigma_i', \sigma_{-i}) - u_i(\sigma))$, exploitability = NashConv/2

출처: ASWISS_CONVERGENCE_KO.md §1. $N =$ 정보집합 수, $N_A = \sum_I |A(I)|$, $\Delta_I = I$ 아래 보수의 폭.


A1. 기호와 정의 (본문 방법)

반복 i의 전차원 후회 $s_i(L;a) = q_i(L;a) - \sum_k \sigma_k(L;b)\, q_i(L;b)$. $q_i(L;a) = \sum_{k\in\mathcal{K}} \sigma^{\text{sh}}_k(k) \sum_{\ell\in\mathcal{L}_k} \Phi^{\text{sh}}(\ell a_i; b)\, u_i(\ell)$

구간 누적 후회 $W = (s_i\colon t_i \le t < |W|)$. $R^{\text{sh}}_k(a) = \sum_{\ell\in\mathcal{L}_k} s_i(L;a)$. 표본 추정 $\hat{R}^{\text{sh}}_k$ (SSW 샘플링, 방문하지 않으면 0)

bucket 합 $R^{\text{sh}}_B = \sum_{k\in B} R^{\text{sh}}_k$. $\psi(s) = \max(0, \max_a s_a)$

리텀 $\hat{a}_i = \arg\max_a \hat{R}^{\text{sh}}_i(a)$. $\hat{R}^{\text{sh}}_i(\hat{a}_i) > \theta^{\text{sh}}_i$ 이면 $\lambda(t) = \hat{a}_i$ 아니면 $\lambda(t) = \varnothing$ ("없음")

분석 조건 (T) $\theta \le \theta^{\text{sh}}_i \le \Delta_R w(W)$, $\kappa(W) = \kappa(|W|)$, 예: 검산 점점 $\kappa\sqrt{\sum_{t=1}^{T}} \le \kappa\Delta_R\sqrt{|W|}$

분할 전경적 합의 리텀이 두 층화 이상인 bucket을 리텀했로 나눈다. 각식은 $Q = 0$, 탐금 표 이에서 시작, 생합 없음

세 정 $C^{\text{sh}} = \sum_k \psi(R^{\text{sh}}_k)$. $R^{\text{sh}}_{\text{sh}} = \sum_{k\in\mathcal{K}} \psi(R^{\text{sh}}_k) - \psi(R^{\text{sh}}_{\text{sh}})$. $\nu_i(t) = \psi(R^{\text{sh}}_i) - R^{\text{sh}}_i(t) \ge 0$

학습기 bucket위값 RM++ $Q_{t+1} = (Q_t, \nu + \tilde{\nu}(R))^+$, $\sigma\nu(R) \propto \sqrt{q_k}$.

가정 (S1) 동시 갱신. (S2) $\mathbb{E}[\ell_t \mid \mathcal{F}_{t-1}] = \nu_t$, $|\ell_t(L;a)| \le \Delta_\ell$. (S3) $\|\hat{\nu}_t(R)\|_\infty \le D_R$. 구간은 두 배액 $\ell_{k,t+} \le \tau\, t_{k,t}$, $\tau = 2$

출력 $\hat{a}^{\text{sh}}$: 구간 종연 UBuy된 전략들의 실현 계획 금등 평균 **NashConv** $= \sum_i (\max_{a'_i} u_i(\sigma'_i, \sigma_{-i}) - u_i(\sigma))_+$ exploitability $=$ NashConv/2

출처: ASWISS_CONVERGENCE_KO.md 상1. $N =$ 정보집합 수, $\bar{r}_{i,t} = \sum_{\tau\le t} \mathbb{1}[A(\tau), \Delta_i = I$ 이면 보수의 목.

본문 방법의 정확한 정의입니다. 라벨은 구간 동안 반복 전략들의 반사실 후회를 더한 값으로 매기고, 표본이면 문턱을 넘을 때만 줍니다. 문턱은 구간 길이보다 느리게 커지기만 하면 되고, 실험에 쓴 정규 검정이 이를 만족합니다. 증명은 동시 갱신과 균등 평균을 가정합니다.

A2. 보조정리

보조정리 1 (NashConv 분해) · (S1) 아래 $\mathrm{NashConv}(\bar{\sigma}^W) \le \frac{1}{|W|}\sum_I \psi(R_I^W) = \frac{C^W + H^W}{|W|}$

보조정리 2 (라벨 항등식) · $H_B^W = \min_{\ell \in A_B \cup \{\varnothing\}} \sum_{I \in B} e_I(\ell)$. 그래서 $H_B = 0 \Leftrightarrow$ 모든 멤버가 함께 최대로 하는 라벨이 있다

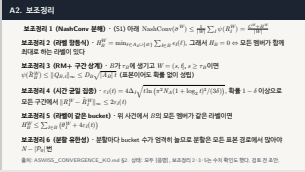
보조정리 3 (RM+ 구간 상계) · B가 τ_B에 생기고 $W = (s, t]$, $s \ge \tau_B$이면 $\psi(\tilde{R}_B^W) \le \|Q_{B,t}\|_\infty \le D_B\sqrt{|A_B|\,t}$ (표본이어도 확률 없이 성립)

보조정리 4 (시간 균일 집중) · $\varepsilon_I(t) = 4\Delta_I\sqrt{t \ln\left(\pi^2 N_A (1+\log_2 t)^2/(3\delta)\right)}$. 확률 $1-\delta$ 이상으로 모든 구간에서 $\|R_I^W - \tilde{R}_I^W\|_\infty \le 2\varepsilon_I(t)$

보조정리 5 (라벨이 같은 bucket) · 위 사건에서 B의 모든 멤버가 같은 라벨이면 $H_B^W \le \sum_{I \in B}\left(\theta_I^W + 4\varepsilon_I(t)\right)$

보조정리 6 (분할 유한성) · 분할마다 bucket 수가 엄격히 늘므로 분할은 모든 표본 경로에서 많아야 $N - |\mathcal{P}_0|$번

출처: ASWISS_CONVERGENCE_KO.md §2. 상태: 모두 [증명], 보조정리 2·3·5는 수치 확인도 했다. 검토 전 초안.


A2. 보조정리

보조정리 1 (NashConv 분해) · (S1) 아래 $\mathrm{NashConv}(\pi^W) \le \frac{1}{1-\gamma}\sum_s \psi(R_1^W) = \frac{\sqrt{|\mathcal{I}|}\,M^W}{1-\gamma}$

보조정리 2 (라벨 점증성) · $R_{2t}^W = \min_{a\in A_{s(t)}(\pi)} \sum_{k\le t} r_k(I)$. 그래서 $R_{2t}=0 \Leftrightarrow$ 모든 행버가 합계 최대로 하는 라벨이 있다

보조정리 3 (RM+ 구간 상계) · 임계 τ_R에 생기고 $W=\{s,\ell_s\times\geq\tau_R$이면 $v(\bar{R}_1^W) \le \|Q_{s,\ell_s}\|_\infty \le D_{\mathrm{KL}}\sqrt{|A_s|}\,\tau$ (표본이어도 확률 얼마 상법)

보조정리 4 (시간 균일 집중) · $\varepsilon_t(\delta) = 4\Delta_1\sqrt{\ln(\pi^2 N_A(1+\log_2 t)^2/(3\delta))}$. 확률 $1-\delta$ 이상으로 모든 구간에서 $|R_1^W - \hat{R}_1^W|_\infty \le 2\varepsilon_t(\delta)$

보조정리 5 (라벨의 같은 bucket) · 위 사건에서 B의 모든 행버가 같은 라벨이면 $M_B^W \le \sum_{I\in B}(W_I^T + 4\varepsilon_t(\delta))$

보조정리 6 (분할 유한성) · 분할마다 bucket 수가 엄격히 늘므로 분할은 모든 표본 경로에서 많아야 $N-|\mathcal{P}_0|$ 번

출처: ASWISS_CONVERGENCE_KO.md §2. 상태: 모두 [증명]. 보조정리 2·3·5는 수치 확인도 했다. 검토 전 주전.

정리 1은 이 여섯 보조정리를 잇습니다. 분해가 NashConv를 bucket 후회와 상쇄로 나누고, RM+가 bucket 후회를 묶고, 집중 부등식이 표본 오차를 묶고, 라벨 항등식이 라벨이 같은 bucket의 상쇄를 묶습니다. 마지막으로 분할이 유한하므로 언젠가 분할이 없는 구간만 남습니다.

A3. 정리 1의 정확한 진술과 따름 결과

정리 1 · (S1)–(S3), (T), 두 배 구간을 가정한다. 확률 $1-\delta$ 이상으로, 끝에서 분할이 없었던 모든 구간 k에서

$$\mathrm{NashConv}(\bar{\sigma}^{(k)}) \le \frac{1}{L_k}\Big[\sum_{B\in\mathcal{P}_k} D_B\sqrt{|A_B|\,t_{k+1}} + \sum_I \big(\theta_I^{(k)} + 6\varepsilon_I(t_{k+1})\big)\Big] = O\Big(\sqrt{\tfrac{\log(kN_A/\delta)}{L_k}}\Big).$$

마지막 분할 구간 $k^* < \infty$ 이후 모든 구간에 적용되고, 거의 확실히 $\mathrm{NashConv}(\bar{\sigma}^{(k)}) \to 0$이다.

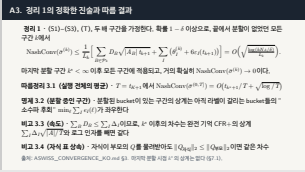
따름정리 3.1 (실행 전체의 평균) · $T = t_{K+1}$에서 $\mathrm{NashConv}(\bar{\sigma}^{(0,T]}) = O\big(t_{k^*+1}/T + \sqrt{\log/T}\big)$

명제 3.2 (분할 중인 구간) · 분할된 bucket이 있는 구간의 상계는 아직 라벨이 갈리는 bucket들의 "소수파 후회" $\min_\ell \sum_I e_I(\ell)$가 좌우한다

비고 3.3 (속도) · $\sum_B D_B \le \sum_I \Delta_I$이므로, k^* 이후의 차수는 완전 기억 CFR+의 상계 $\sum_I \Delta_I\sqrt{|A|/T}$와 로그 인자를 빼면 같다

비고 3.4 (자식 표 상속) · 자식이 부모의 Q를 물려받아도 $\|Q_{\text{자식}}\|_2 \le \|Q_{\text{부모}}\|_2$이면 같은 차수

출처: ASWISS_CONVERGENCE_KO.md §3. 마지막 분할 시점 k^*의 상계는 없다 (§7.1).


A3. 정리 1의 정확한 진술과 따름 결과

정리 1 · (S1)–(S3), (T), 두 배 구간을 가정한다. 확률 $1-\delta$ 이상으로, 끝에서 분할이 없었던 모든 구간 i에서

$$\mathrm{NashConv}(\sigma^{(2k)}) \le \frac{1}{T_k}\Big(\sum_{R\in\mathcal{P}_k} D_R\sqrt{|A_R|\,b_{k+1}} + \sum_t \langle \theta_t^{(k)}, \ell_t\rangle\Big) = O\Big(\sqrt{\tfrac{|\mathrm{Info}|\,\log|\mathrm{Info}|}{T_k}}\Big).$$

마지막 분할 구간 $k^\star<\infty$ 이후 모든 구간에 적용되고, 거의 확실히 $\mathrm{NashConv}(\sigma^{(k)})\to 0$ 이다.

따름정리 3.1 (실행 전체의 평균) · $T=t_{K+1}$ 에서 $\mathrm{NashConv}(\bar\sigma^{(K,T)}) = O\big(t_{K+1}/T + \sqrt{\log T/T}\big)$

명제 3.2 (분할 중인 구간) · 분할된 bucket이 있는 구간의 상계는 아직 라벨이 갈리는 bucket들의 소수의 후회 $\sum_{r\in U}\mathrm{reg}_r/T$로 지배된다.

비교 3.3 (속도) · $\sum_k D_k \le \sum_k \Delta_i$이므로, $k^\star$ 이후의 차수는 완전 기억 CFR+의 상계 $\sum_i \Delta_i\sqrt{|\mathcal{A}|/T}$와 로그 인자를 빼면 같다.

비교 3.4 (자식 표 상속) · 자식이 부모의 상를 물려받아도 $\|Q_{\mathrm{child}}\|_2 \le \|Q_{\mathrm{par}}\|_2$이면 같은 차수

출처: ASWISS_CONVERGENCE_KO.md §3. 마지막 분할 시점 $k^\star$의 상계는 없다(§7.1).

정리 1의 정확한 식입니다. 첫 항은 RM+가 묶는 bucket 후회, 둘째 항은 문턱과 표본 오차입니다. 둘 다 구간 길이의 제곱근 차수라서, 구간 길이로 나누면 제곱근에 반비례해 줄어듭니다. 마지막 분할 이후에는 완전 기억 CFR+와 같은 차수입니다. 분할이 진행 중인 구간에서는 아직 라벨이 갈리는 bucket의 소수파 후회가 오차를 좌우합니다.

A4. 정리 1이 보장하지 않는 것

1. **마지막 분할 시점에 상계가 없다.** CFR+IRA도 이를 직접 구하지 못했다(원문의 $T_{\lim}$은 정보집합 수에 지수적). 후보: 상위 두 행동의 누적 후회 차가 $\Omega(L)$인 여유 조건
2. **최종 bucket 수에 보장이 없다.** 고정 수준의 z 검정은 거짓 라벨이 무한히 자주 나올 수 있어 원리적으로 정보집합마다 하나까지 갈 수 있다(수렴은 유지). 여유 라벨 집합(A5)은 좋은 사건 위에서 거짓 분할을 없앤다
3. **학습 메모리를 줄이지 않는다.** 정보집합마다 구간 누적값($|\mathcal{I}| \cdot |A|$)을 둔다. 저장 전략 재생(방법 A)은 이를 bucket 표 K개로 바꾸는 대신 속도를 잃는다
4. **보장 대상은 정보집합별 평균이다.** bucket 표에 압축한 평균에는 보장이 없다(mini-stud의 한 실행에서는 가까웠다: 0.0216 대 0.0198)
5. **가정과 구현이 다르다.** 정리는 동시 갱신과 구간 균등 평균, 구현은 교대 갱신과 선형 평균
6. **충분조건일 뿐이다.** 최선 응답이 가지 않는 정보집합의 후회도 상쇄를 키워 불필요한 분할을 낳을 수 있다(정확성 손해는 없고 효율 손해만)

출처: ASWISS_CONVERGENCE_KO.md §7.1–7.2.


A4. 정리 1이 보장하지 않는 것

1. 마지막 분할 시점에 상계가 없다. $CFR_T = IRA$도 이를 직접 구하지 못했다(원문의 T_{ex}은 정보집합 수에 지수적). 후보: 상위 두 랜덤의 누적 후회 차가 $O(J)$인 여유 조건.
2. 최종 bucket 수에 로그항이 없다. 고정 수준의 ε 검정은 거장 라벨이 무한히 자주 나올 수 있어 원리적으로 정보집합(IC) 하나하나 잘 수 있다(수정은 유지). 여유 라벨 집합 $(A1)$은 좋은 사건 위에서 거장 분할을 없앤다.
3. 학습 헤로리를 줄여지 않는다. 정보집합마다 구간 누적값 $(U_i^+ - U_i^-)$을 둔다. 저경 전략 재생(헤밀 A)은 이를 bucket 표시 개념 바꾸는 대신 속도를 빌린다.
4. 보장 대상은 정보집합별 평균이다. bucket 표에 압축한 평균에는 보장이 없다 (mini-study의 한 실행에서는 가까웠다: 0.0216 대 0.0180).
5. 거장과 구현이 다르다. 정리는 둘시 경신과 구간 균등 평균, 구현은 과대 경신과 선형 평균
6. 충분조건일 뿐이다. 최선 응답이 거지 않는 정보집합의 후회도 상태를 커워 불필요한 분할을 낳을 수 있다 (정확성 손해는 없고 효율 손해만)

출처: ASWISS_CONVERGENCE_KO.md §7.1–7.2.

정리 1이 말하지 않는 것을 모았습니다. 가장 중요한 것은 마지막 분할 시점에 상계가 없다는 것과, 학습 메모리를 줄이지 않는다는 것입니다. 또 보장하는 정책은 정보집합별 평균이고, 실제 구현은 정리의 가정과 조금 다릅니다. 이 한계들은 논문에서도 그대로 밝힐 내용입니다.

A5. 잡음 아래의 라벨: 여유 라벨 집합과 저장 전략 재생

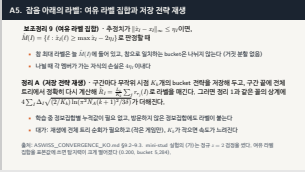
보조정리 9 (여유 라벨 집합) · 추정치가 $\|\hat{x}_I - x_I\|_\infty \le \eta_I$이면, $\hat{M}(I) = \{\ell : \hat{x}_I(\ell) \ge \max \hat{x}_I - 2\eta_I\}$로 판정할 때

- 참 최대 라벨은 늘 $\hat{M}(I)$에 들어 있고, 참으로 일치하는 bucket은 나뉘지 않는다 (거짓 분할 없음)
- 나뉠 때 각 멤버가 가는 자식의 손실은 $4\eta_I$ 이내다

정리 A (저장 전략 재생) · 구간마다 무작위 시점 K_k개의 bucket 전략을 저장해 두고, 구간 끝에 전체 트리에서 정확히 다시 계산해 $\hat{R}_I = \frac{L_k}{K_k}\sum_j r_{\tau_j}(I)$로 라벨을 매긴다. 그러면 정리 1과 같은 꼴의 상계에 $4\sum_I \Delta_I \sqrt{(2/K_k)\ln(\pi^2 N_A (k+1)^2/3\delta)}$가 더해진다.

- 학습 중 정보집합별 누적값이 필요 없고, 방문하지 않은 정보집합에도 라벨이 붙는다
- 대가: 재생에 전체 트리 순회가 필요하고(작은 게임만), K_k가 작으면 속도가 느려진다

출처: ASWISS_CONVERGENCE_KO.md §9.2–9.3. mini-stud 실험의 (가)는 정규 $z = 2$ 검정을 썼다. 여유 라벨 집합을 표본값에 쓰면 탐지력이 크게 떨어졌다 (0.200, bucket 5,284).


A5. 잡음 아래의 라벨: 여유 라벨 집합과 저장 전략 재생

보조정리 9 (여유 라벨 집합) · 추정치가 $\|\hat{s}_k - s\|_\infty \le s_0$이면, $\mathcal{M}(\hat{s}) = \{\ell : \hat{s}_\ell(\ell) \ge \max_{k \ne \ell} \hat{s}_k - 2s_0\}$로 정의할 때

- 참 최대 라벨은 늘 $\mathcal{M}$에 들어 있고, 참으로 절치하는 bucket은 나머지 많아야 (거듭 분할 없음)
- 나빌 때 각 멤버가 가는 자식의 손실은 $4s_0$ 이내다

정리 A (저장 전략 재생) · 구간마다 무작위 시점 K_j개의 bucket 전하을 저장해 두고, 구간 끝에 전체 트리에서 정확히 다시 계산해 $\hat{s}_j = \frac{1}{K_j}\sum_{i=1}^{K_j} r_{\tau_i}(\ell)$로 라벨을 매긴다. 그러면 정리 1과 같은 꼴의 상계에 $4\sum_j \Delta_{\tau_j}\sqrt{(2/K_j)\ln(\pi^2 N_j(K+1)^2/3\delta)}$가 더해진다.

- 학습 중 경로합계별 누적값이 필요 없고, 방문하지 않은 경로합계에도 라벨이 붙는다
- 대가: 재생에 전체 트리 순회가 필요하고(작은 게임만), K_j가 작으면 속도가 느려진다

출처: ASWISS_CONVERGENCE_KO.md §A.2–A.3. mini-max 상합의 (가)는 정규 $\varepsilon = 2$ 경합을 뜻다. 여유 라벨 집합을 표준경계 수만 방지해야 고개 멈어진다 (0.200, bucket 5.2배).

표본 잡음이 있을 때 라벨을 어떻게 매기는지에 대한 두 결과입니다. 여유 라벨 집합은 잡음 폭 안의 행동을 모두 후보로 두어 거짓 분할을 없앱니다. 저장 전략 재생은 정보집합별 누적값을 저장하지 않고 몇 개의 전략만 저장했다가 정확히 다시 계산하는 방법으로, 정리 1과 같은 꼴의 보장이 있습니다. 다만 전체 트리 순회가 필요해 작은 게임에서만 그대로 쓸 수 있습니다.

A6. 실험 설정과 비교 조건의 차이

	mini-stud	Kuhn · Leduc
게임	정보집합 51,114	Kuhn 12, Leduc 288 (rank만)
학습기	외부 샘플링 CFR+(교대 갱신, 선형 평균)	정확한 전체 트리 CFR+
학습량	1.638억 반복 (규칙 비교는 4,096만)	65만 5,350반복
구간	1만 반복에서 두 배씩, 자식은 표를 0에서	10반복에서 두 배씩
라벨	정규 검정 $z = 2$	도달 확률당 후회가 10^{-3} 보다 클 때
출발점 · seed	time step 40개 bucket, seed 1	$\mathcal{P}_0$ + 무작위 추상화 20개 (트리 10, 자유 10)
지표	bucket 평균 정책의 정확한 exploitability	같음
기준선	완전 기억, 카드 k-means 7,051 (bucket 수 다름), time step 고정	추상화 없음, 실행마다 최종 bucket 수에 맞춘 k-means

출처: aswiss/README_KO.md, aswiss/small/README_KO.md, small/run_leduc_infoset.sh. mini-stud 비교군의 최종 bucket 수가 서로 다르다 (본문 26,844, B 14,910, k-means 7,051). seed가 하나라 mini-stud의 작은 차이는 같은 값으로 읽는 것이 맞다.

A6. 실험 설정과 비교 조건의 차이

	mini-stud	Kuhn · Leduc
게임	정보집합 11,114	Kuhn 12, Leduc 288 (rank만)
학습기	외부 샘플링 CFR+ (교대 갱신, 선형 할인)	정확한 전체 트리 CFR+
학습량	1,638억 반복 (교착 비교는 4,096만)	65만 5,360만복
규칙	1인 반복에서 6 배액, 자식은 표를 이해서	10만복에서 6 배액
라벨	정규 검정 $z = 2$	도달 확률당 충화가 10^{-12} 모다 클 때
출발점 · seed	time step 4개 bucket, seed 1	P_1 + 무작위 추상화 20개(트리 10, 자유 10)
지표	bucket 평균 정책의 경확한 exploitability	같음
기준선	완전 기억, 커도 k-means 7,051 (bucket 수 다름), time step 고정	추상화 없음, 실행마다 최종 bucket 수에 맞춘 k-means

출처: assets/README_KO.md, assets/excel/README_KO.md, stud/run_leduc_cfrconf.sh, mini-stud 비교군의 최종 bucket 수가 서로 다르다 (본문 26,844, 표 14,910, k-means 7,051). seed가 하나만 mini-stud의 직관 차이는 같은 값으로 읽는 것이 맞다.

실험별 설정입니다. mini-stud는 외부 샘플링, Kuhn과 Leduc은 정확한 CFR+입니다. mini-stud의 k-means 기준선은 bucket 수가 본문 방법과 달라서 같은 조건의 비교가 아니고, Leduc의 k-means는 각 실행의 최종 bucket 수에 맞췄습니다. mini-stud는 seed가 하나라는 점도 기억해 주십시오.

T1. 평균 전략 라벨 (방법 B)

정의 · 판정 시점의 평균 전략 $\bar{\sigma}$을 고정하고, 그 한 프로필 아래의 순간 반사실 후회 $r^{\bar{\sigma}}(I)$로 라벨을 매긴다. 저장한 $\bar{\sigma}$ 하나로 처음 보는 정보집합의 라벨도 계산할 수 있다.

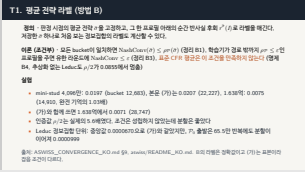
이론 (조건부) · 모든 bucket이 일치하면 $\mathrm{NashConv}(\bar{\sigma}) \le \rho_{\mathcal{P}}(\bar{\sigma})$ (정리 B1). 학습기가 경로 밖까지 $\rho_{\mathcal{P}} \le \varepsilon$인 프로필을 주면 유한 라운드에 $\mathrm{NashConv} \le \varepsilon$ (정리 B3). 표준 CFR 평균은 이 조건을 만족하지 않는다 (명제 B4, 추상화 없는 Leduc도 $\rho/2$가 0.0855에서 멈춤)

실험

- mini-stud 4,096만: 0.0197 (bucket 12,683), 본문 (가)는 0.0207 (22,227). 1.638억: 0.0075 (14,910, 완전 기억의 1.03배)
- (가)와 함께 쓰면 1.638억에서 0.0071 (28,747)
- 인증값 $\rho/2$는 실제의 5.6배였다. 조건은 성립하지 않았는데 분할은 좋았다
- Leduc 정보집합 단위: 중앙값 0.0000670으로 (가)와 같았지만, $\mathcal{P}_0$ 출발은 65.5만 반복에도 분할이 이어져 0.0000999

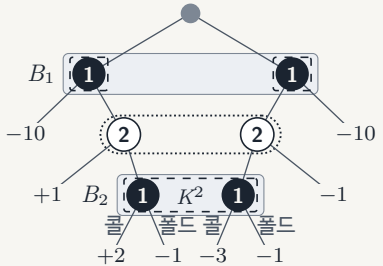
출처: ASWISS_CONVERGENCE_KO.md §9, aswiss/README_KO.md. B의 라벨은 정확값이고 (가)는 표본이라 잡음 조건이 다르다.

└T1. 평균 전략 라벨 (방법 B)



첫 번째 트릭은 평균 전략 라벨입니다. 판정할 때 평균 전략을 하나 고정해 두고 그 아래에서 후회를 계산합니다. 저장한 전략 하나로 처음 보는 정보집합의 라벨도 계산할 수 있어서 큰 게임의 소속 문제에 쓸 만한 후보입니다. 실험에서는 비슷한 값에 bucket이 절반 남짓이었습니다. 다만 이론은 학습기가 경로 밖까지 후회를 줄여야 한다는 조건이 붙고, 표준 CFR 평균은 그 조건을 만족하지 않습니다. 그래서 본문이 아니라 트릭으로 둡니다.

T2. bucket 이력 키와 이력 안 상쇄 (명제 H4)



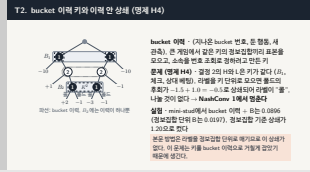
파선: bucket 이력. B_2에는 이력이 하나뿐

bucket 이력 · (지나온 bucket 번호, 둔 행동, 새 관측). 큰 게임에서 같은 키의 정보집합끼리 표본을 모으고, 소속을 번호 조회로 정하려고 만든 키

문제 (명제 H4) · 결정 2의 H와 L은 키가 같다 (B_1, 체크, 상대 베팅). 라벨을 키 단위로 모으면 폴드의 후회가 $-1.5 + 1.0 = -0.5$로 상쇄되어 라벨이 "콜", 나눌 것이 없다 → **NashConv 1에서 멈춘다**

실험 · mini-stud에서 bucket 이력 + B는 0.0896 (정보집합 단위 B는 0.0197). 정보집합 기준 상쇄가 1.20으로 컸다

본문 방법은 라벨을 정보집합 단위로 매기므로 이 상쇄가 없다. 이 문제는 키를 bucket 이력으로 거칠게 잡았기 때문에 생긴다.


T2. bucket 이력 키와 이력 안 상쇄 (명제 H4)
J_0
J_1
-10
$+10$
$+1$
-1
-10
$+10$
$+1$
-1
-5
$+5$
-5
$+5$
$+1$
-1
-1
$+1$
-1
$+1$
-1
$+

T3. 역방향 분할 (휴리스틱)

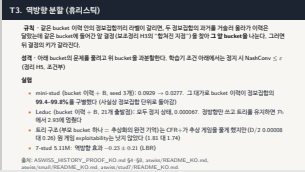
규칙 · 같은 bucket 이력 안의 정보집합끼리 라벨이 갈리면, 두 정보집합의 과거를 거슬러 올라가 이력은 달랐는데 같은 bucket에 들어간 앞 결정(보조정리 H3의 "합쳐진 지점")을 찾아 **그 앞 bucket을** 나눈다. 그러면 뒤 결정의 키가 갈라진다.

성격 · 아래 bucket의 문제를 풀려고 위 bucket을 과분할한다. 학습기 조건 아래에서는 정지 시 NashConv $\leq \varepsilon$ (정리 H5, 조건부)

실험

- mini-stud (bucket 이력 + B, seed 3개): 0.0929 → 0.0277. 그 대가로 bucket 이력이 정보집합의 **99.4–99.8%**를 구별했다 (사실상 정보집합 단위로 돌아감)
- Leduc (bucket 이력 + B, 21개 출발점): 모두 정지 상태, 0.000067. 정방향만 쓰고 트리를 유지하면 $\mathcal{P}_0$에서 2.93에 멈췄다
- 트리 구조(부모 bucket 하나 = 추상화의 완전 기억)는 CFR+가 추상 게임을 풀게 했지만(D/2 0.00008 대 0.26) 원 게임 exploitability는 낫지 않았다 (1.81 대 1.74)
- 7-stud 5.11M: 역방향 효과 -0.23 ± 0.21 (LBR)

출처: ASWISS_HISTORY_PROOF_KO.md §4·§8, aswiss/README_KO.md, aswiss/small/README_KO.md, aswiss/stud7/README_KO.md.



T3. 역방향 분할 (휴리스틱)

규칙 · 같은 bucket 이력 안의 정보집합끼리 라벨이 갈리면, 두 정보집합의 끝키를 거슬러 올라가 이력은 달랐는데 같은 bucket에 들어간 앞 결정 (보조정리 H1의 "합계전 저점")을 찾아 그 쌍 bucket을 나눕다. 그러면 뒤 결정의 키가 갈라진다.

성격 · 이력 bucket의 문제를 풀려고 위 bucket을 과분할한다. 학습기 조건 아래에서는 경지 시 NashConv $\leq \varepsilon$ (정리 H5, 조건부)

설명

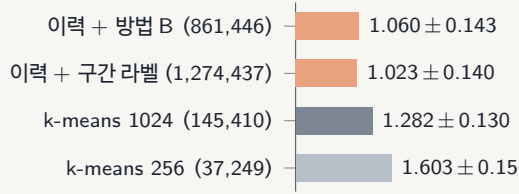
- mini-stud (bucket 이력 ≈ 8, seed 3개): 0.0929 → 0.0277. 그 대가로 bucket 이력이 정보집합의 **99.4–99.8%**를 구별한다 (사실상 정보집합 단위로 돌아감)
- Leduc (bucket 이력 ≈ 8, 21개 출발점): 모두 정지 상태, 0.000067. 경험합인 쓰고 트리를 유지하면 7h에서 2.93배 멈췄다
- 트리 구조(부모 bucket 하나 ≈ 추상화의 완전 기억)는 CFR+가 추상 게임을 풀게 했지만 (D/2 0.00008 대 0.26) 동 게임 exploitability는 넛지 않았다 (1.81 대 1.74)
- 7-stud 5.11M: 역방향 효과 −0.23 ± 0.21 (LBR)

출처: ASWISS_HISTORY_PROOF_KO.md §4-§8, aswiss/README_KO.md, aswiss/kuhn3/README_KO.md, aswiss/stud7/README_KO.md.

세 번째 트릭은 역방향 분할입니다. bucket 이력 키가 보지 못하는 차이를 드러내려고, 앞 결정의 bucket을 나눠 뒤 결정의 키를 갈라 놓는 장치입니다. 아래 bucket의 문제를 위해 위 bucket을 과분할하는 셈이고, mini-stud에서는 결국 이력이 정보집합의 99퍼센트 이상을 구별하게 되어 사실상 정보집합 단위로 돌아갔습니다. 그래서 본문 방법, 즉 처음부터 정보집합 단위로 그 자리에서 나누는 쪽이 더 직접적인 해법이라고 봅니다.

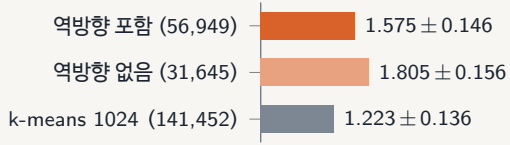
T4. 7-stud (원자와 bucket 이력을 쓴 실행)

100M roots · 역방향 없음



방법 B − k-means 1024 = -0.22 $[-0.41, -0.04]$
(같은 deal로 짝지은 차이)

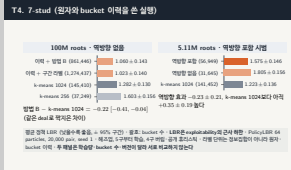
5.11M roots · 역방향 포함 시범



역방향 효과 -0.23 ± 0.21. k-means 1024보다 아직 $+0.35 \pm 0.19$ 높다

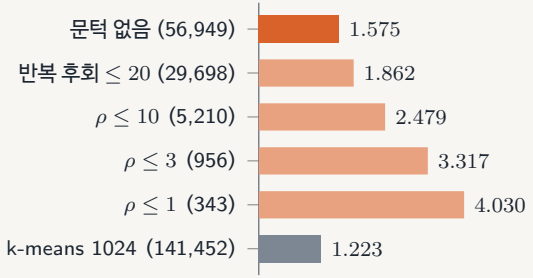
평균 정책 LBR (낮을수록 좋음, ± 95% 구간) · 괄호: bucket 수 · **LBR은 exploitability의 근사 하한** · PolicyLBR 64 particles, 20,000 pair, seed 1 · 헤즈업, 5구부터 학습, 4구 버림·공개 휴리스틱 · 라벨 단위는 정보집합이 아니라 원자·bucket 이력 · **두 패널은 학습량·bucket 수·버전이 달라 서로 비교하지 않는다**

└T4. 7-stud (원자와 bucket 이력을 쓴 실행)



7-stud 결과는 모두 트릭을 쓴 실행이라 부록으로 옮겼습니다. 1억 roots 실행에서는 bucket 이력과 평균 전략 라벨을 쓴 비교군이 k-means 1024보다 LBR이 낮았고, 511만 roots 시범에서는 역방향 분할이 LBR을 낮췄지만 아직 k-means 1024보다 높았습니다. 라벨 단위가 정보집합이 아니므로 본문 정리의 검증은 아닙니다.

T5. 판정 문턱 ε (학습기 조건 흉내)



7-stud 5.11M roots (bucket 이력 + B + 역방향) · 평균 정책 LBR, 20,000 pair · 괄호: 마지막 bucket 수

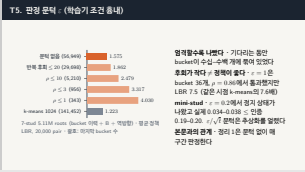
엄격할수록 나빴다 · 기다리는 동안 bucket이 수십–수백 개에 묶여 있었다

후회가 작다 $\neq$ 정책이 좋다 · $\varepsilon = 1$은 bucket 36개, $\rho = 0.86$에서 통과했지만 LBR 7.5 (같은 시점 k-means의 7.6배)

mini-stud · $\varepsilon = 0.2$에서 정지 상태가 나왔고 실제 0.034–0.038 $\leq$ 인증 0.19–0.20. $\varepsilon/\sqrt{t}$ 문턱은 추상화를 얼렸다

본문과의 관계 · 정리 1은 문턱 없이 매 구간 판정한다

└T5. 판정 문턱 ε (학습기 조건 휴내)



판정 문턱은 평균 전략 라벨의 이론, 즉 학습기 조건을 실험에서 흉내 내려고 넣은 장치입니다. 7-stud에서는 엄격할수록 나빠졌고, 후회가 작다고 정책이 좋은 것도 아니었습니다. 본문 정리 1은 문턱 없이 매 구간 판정하는 방식이므로 이 장치가 필요 없습니다.

T6. 혼합 평균 (평균 전략 라벨의 경로 밖 문제)

문제 · 평균 전략은 자기가 거의 가지 않게 된 곳에서 초반 값에 멈춘다. 정책에는 영향이 없지만 (실현 계획의 질량이 0), 자주 가는 정보집합의 라벨이 그 아래의 멈춘 값으로 계산되어 분할 신호를 놓친다.

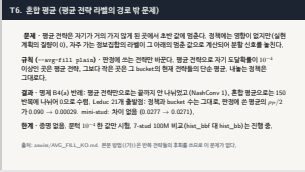
규칙 (`--avg-fill plain`) · 판정에 쓰는 전략만 바꾼다. 평균 전략으로 자기 도달확률이 10^{-4} 이상인 곳은 평균 전략, 그보다 작은 곳은 그 bucket의 현재 전략들의 단순 평균. 내놓는 정책은 그대로다.

결과 · 명제 B4(a) 반례: 평균 전략만으로는 끝까지 안 나뉘었고 (NashConv 1), 혼합 평균으로는 150 반복에 나뉘어 0으로 수렴. Leduc 21개 출발점: 정책과 bucket 수는 그대로, 판정에 쓴 평균의 $\rho_{\mathcal{P}}/2$ 가 0.090 $\rightarrow$ 0.00029. mini-stud: 차이 없음 (0.0277 $\rightarrow$ 0.0271).

한계 · 증명 없음. 문턱 10^{-4} 한 값만 시험. 7-stud 100M 비교 (hist_bbf 대 hist_bb) 는 진행 중.

출처: aswiss/AVG_FILL_KO.md. 본문 방법 ((가)) 은 반복 전략들의 후회를 쓰므로 이 문제가 없다.

└T6. 혼합 평균 (평균 전략 라벨의 경로 밖 문제)



마지막 트릭은 혼합 평균입니다. 평균 전략 라벨에서만 생기는 경로 밖 문제를 피하는 장치입니다. 본문 방법은 반복 전략들의 후회를 쓰므로 이 문제가 없습니다.